

DG INTERNS HUB

Cybersecurity Internship

WEEK 8 ASSIGNMENT

ADVANCED VAPT – ATTACK CHAIN & FULL EXPLOITATION PROJECT

Submitted By	Akash Sanuria
Intern ID	DG/SEPTEMBER/CYBER/171
Internship	Cybersecurity Internship
Week	Week 8
Assignment	Advanced VAPT – Attack Chain & Full Exploitation
Target	OWASP Juice Shop (Local Lab)
Report Date	28 October 2026

Contents and Evidence Register

Report structure

1. Executive Summary; 2. Scope and Authorization; 3. Laboratory Environment; 4. Attack Surface Mapping; 5. Testing Methodology; 6. Finding A – SQL Injection; 7. Finding B – Administrative Access; 8. Finding C – Basket Authorization; 9. Attack Chain Interpretation; 10. Risk and Remediation; 11. Validation and Limitations; 12. Evidence Appendix.

Evidence E01

SQL injection login request to POST /rest/user/login and successful HTTP 200 administrator response. File: 01_SQL_Injection_Login_Bypass.png.

Evidence E02

Administration page visible after administrator login. File: 02_Admin_Section_Access.png.

Evidence E03

Regular user identity response from GET /rest/user/whoami, showing user ID 25. File: 04_Zara_Authentication.png.

Evidence E04

Basket resource response from GET /rest/basket/1, showing basket owner UserId 1. File: 05_Normal_User_IDOR_Confirmed.png.

Evidence handling

All report screenshots must be original captured images with bearer tokens, cookies and passwords redacted. AI-recreated screenshots should not be treated as primary forensic evidence.

1. Executive Summary

Project overview

This report presents a controlled vulnerability assessment of the OWASP Juice Shop application hosted on a local test system. The assignment objective was to examine application entry points, test security controls, and explain how weaknesses could combine into an attack scenario.

Principal result: authentication

A crafted value submitted to the login endpoint produced a successful authentication response for the administrator account (user ID 1). The observed HTTP 200 response and returned administrator identity support a finding of SQL injection-based authentication bypass in this intentionally vulnerable training application.

Principal result: privileged access

After the login bypass, the tester opened the Administration interface. The page displayed registered users and customer feedback. This demonstrates impact from the compromised administrator session; it does not establish that the administration interface itself was independently bypassed.

Principal result: object access

In a separate regular-user session, GET /rest/user/whoami returned Zara, user ID 25. A request for basket 1 returned an object owned by user ID 1. This supports a cross-user basket access concern if the same Zara authentication context was maintained across both requests.

Overall conclusion

The exercise provides confirmed evidence for the login bypass and subsequent administrator access, plus conditional evidence for broken object-level authorization. Findings and limitations are separated to avoid overstating what the screenshots prove.

2. Scope, Objectives and Rules of Engagement

Authorized target

OWASP Juice Shop running at http://localhost:3000, accessed through the student's local Kali Linux virtual-machine workflow and browser/Burp testing environment.

Objectives

Map visible login and API entry points; evaluate authentication logic; examine access to privileged application functionality; test whether object identifiers are protected by ownership checks; record reproducible, redacted evidence.

In-scope testing

HTTP request inspection and replay using Burp Suite Community Edition; local browser navigation; read-only verification of account and basket metadata; interpretation of application responses.

Out of scope

No production systems, third-party accounts, external hosts, destructive changes, persistence, or actual data exfiltration are claimed. Screenshots show only the observations included in this report.

Evidence standard

A screenshot supports only the request and response it displays. A successful response does not by itself prove authorization failure unless the caller identity, object ownership and expected access policy are established.

3. Laboratory Environment and Attack Surface

Environment

Target: locally hosted OWASP Juice Shop. Tester platform: Kali Linux VM. Primary interception and request-replay tool: Burp Suite Community Edition, Repeater. Browser used for login and application navigation. Testing evidence dated 28 October 2026.

Authentication entry point

POST /rest/user/login accepts JSON login input and returns an authentication response. The test focused on whether server-side login logic correctly separated input data from database query structure.

Session identity endpoint

GET /rest/user/whoami returns information about the currently authenticated principal. It was used to establish the identity of the regular-user session visible in the screenshot.

Object-level API

GET /rest/basket/{id} retrieves a basket resource by numeric identifier. Its authorization behavior must be evaluated against the authenticated user and the returned UserId owner field.

Administrative interface

The /#/administration route was visited after administrator authentication. The screenshot displays registered users and customer feedback, illustrating the sensitive functionality available to a privileged account.

Potential exposure categories

Authentication bypass, excessive privilege, missing object ownership checks, and sensitive-data exposure are relevant categories for this lab. Only demonstrated outcomes are treated as observed results.

4. Methodology and Reproduction Record

Phase 1 — Baseline mapping

Identify the login request in the browser and inspect it in Burp. Preserve the HTTP method, endpoint, JSON format and target host while observing normal request/response behavior.

Phase 2 — Authentication test

Submit a crafted login email value containing a SQL comment sequence through the training application. Observe the response status and returned account identity. The evidence shows a 200 OK login response for the administrator identity.

Phase 3 — Impact verification

With the authenticated administrator session, open the Administration interface and confirm that privileged data is displayed. Record a separate screenshot of the page.

Phase 4 — Regular-user identity

In a separate session, send GET /rest/user/whoami and record the returned user ID. The provided evidence shows user ID 25.

Phase 5 — Object access

Using the regular-user session, request GET /rest/basket/1. Compare the returned basket owner with the authenticated user ID. The provided response shows basket ID 1 and UserId 1.

Quality controls

Keep the same token for the identity and basket requests when verifying IDOR. Compare unredacted originals locally, then redact secrets before sharing. Do not use modified screenshots to imply actions that did not occur.

5. Finding A — SQL Injection Login Bypass

Finding classification

SQL Injection affecting authentication logic. Severity: Critical in the controlled lab because it resulted in administrator authentication without the intended credential verification.

Affected request

POST /rest/user/login at localhost:3000. The request JSON contains an email field with an injected SQL comment sequence and a password field. The screenshot shows the correct login endpoint.

Observed response

HTTP/1.1 200 OK. The JSON authentication object identifies admin@juice-sh.op and user ID 1. Authentication tokens and the submitted password have been blacked out in the supplied evidence.

Why it matters

If untrusted login input changes the structure of a database query, authentication checks may be bypassed. Successful administrator authentication can expose privileged functions and data.

Root-cause hypothesis

The observed behavior is consistent with unsafe query construction in the application’s login path. Source-code review would be needed to establish the exact vulnerable statement.

Remediation

Use parameterized queries or safe ORM query APIs; avoid concatenating untrusted input into SQL; enforce server-side credential verification; add negative authentication tests; monitor suspicious login patterns.

Evidence

E01: redacted SQL injection login request and administrator authentication response.

6. Finding B — Administration Interface Access

Finding classification

Post-authentication impact of the administrator login bypass. Not a separately confirmed administrative authorization bypass.

Observed result

The Administration screen displays a Registered Users list and Customer Feedback panel. The application was accessed while authenticated as the administrator following the login bypass.

Security relevance

A compromise of an administrative account increases exposure because privileged interfaces may provide user management, content moderation or access to customer-related information.

Evidence boundary

The screenshot demonstrates successful access to the interface, not that an unauthenticated or ordinary user can access it. Treat this as an impact step in the attack chain rather than a distinct vulnerability without additional testing.

Defensive recommendations

Enforce role checks in backend APIs, protect administrative actions with authorization middleware, maintain auditable admin events, apply least privilege, and test unauthorized requests for both pages and APIs.

Evidence

E02: administration interface screenshot with registered users and feedback.

7. Finding C — Cross-User Basket Access

Finding classification

Potential Insecure Direct Object Reference / Broken Object Level Authorization (IDOR/BOLA). Severity: High if the same ordinary-user session is confirmed for both requests.

Identity evidence

GET /rest/user/whoami returned HTTP 200 and user ID 25 for Zara. This establishes the regular-user identity visible in E03.

Object evidence

GET /rest/basket/1 returned HTTP 200, status success, basket ID 1 and UserId 1. The Products array is empty in this capture, so the screenshot does not show disclosure of basket product contents.

Reasoning

The authenticated user ID and basket owner ID differ. If the identical Zara bearer token/cookie was used for both requests, the API exposed metadata about another user’s basket.

Limitation and confirmation requirement

The tokens are correctly redacted in the submitted images; therefore the screenshots alone cannot independently prove token equality. Compare original Burp requests locally before marking this finding fully verified.

Remediation

Enforce a server-side ownership check for every basket lookup; deny unauthorized access with an appropriate status; add integration tests using two separate normal accounts; do not rely on client-supplied object identifiers as authorization.

8. Attack Chain and Risk Assessment

Chain A — Demonstrated sequence

Login endpoint input manipulation → successful administrator authentication → administration interface access. This sequence is supported by E01 and E02 and shows how an authentication weakness can lead to privileged application access.

Chain B — Separate authorization scenario

Regular user authentication (user ID 25) → request for basket 1 → response showing owner user ID 1. This is a separate testing session, not a continuation of the administrator session. It should not be presented as direct lateral movement from the SQL injection.

Risk priority: SQL injection

Critical. Potential confidentiality and integrity impact from unauthorized administrator authentication. Fix authentication query handling first.

Risk priority: object authorization

High if verified. Potential confidentiality impact through cross-user object reads; impact shown in this specific capture is limited to basket metadata.

Risk priority: administration exposure

Impact amplifier rather than independent finding. Protect sensitive functions with robust authorization even when a user possesses a valid session.

Risk communication

Severity values are qualitative lab judgments, not formal CVSS scores. Production severity would depend on deployment context, access controls and demonstrated data exposure.

9. Remediation, Retesting and Evidence Integrity

Immediate fixes

Replace unsafe SQL construction with parameterized statements and regression tests. Centralize object authorization checks and ensure that each basket request is bound to the authenticated owner.

Retest A — Login

Repeat the previously successful login input against the patched application. Expected result: authentication fails without valid credentials, and no administrator session is issued.

Retest B — Administrative functions

Verify that authenticated normal users and unauthenticated clients cannot retrieve administrative data or invoke privileged APIs. Confirm server-side denial, not only client-side route blocking.

Retest C — Basket ownership

Use two ordinary accounts A and B. Confirm A can read its own basket but cannot read B’s basket. Record HTTP status and confirm no unauthorized basket fields are returned.

Evidence handling

Preserve original request history privately; redact passwords, bearer tokens and cookie values on exported screenshots; do not publish active credentials; retain request paths, response codes and relevant IDs.

Reporting limitations

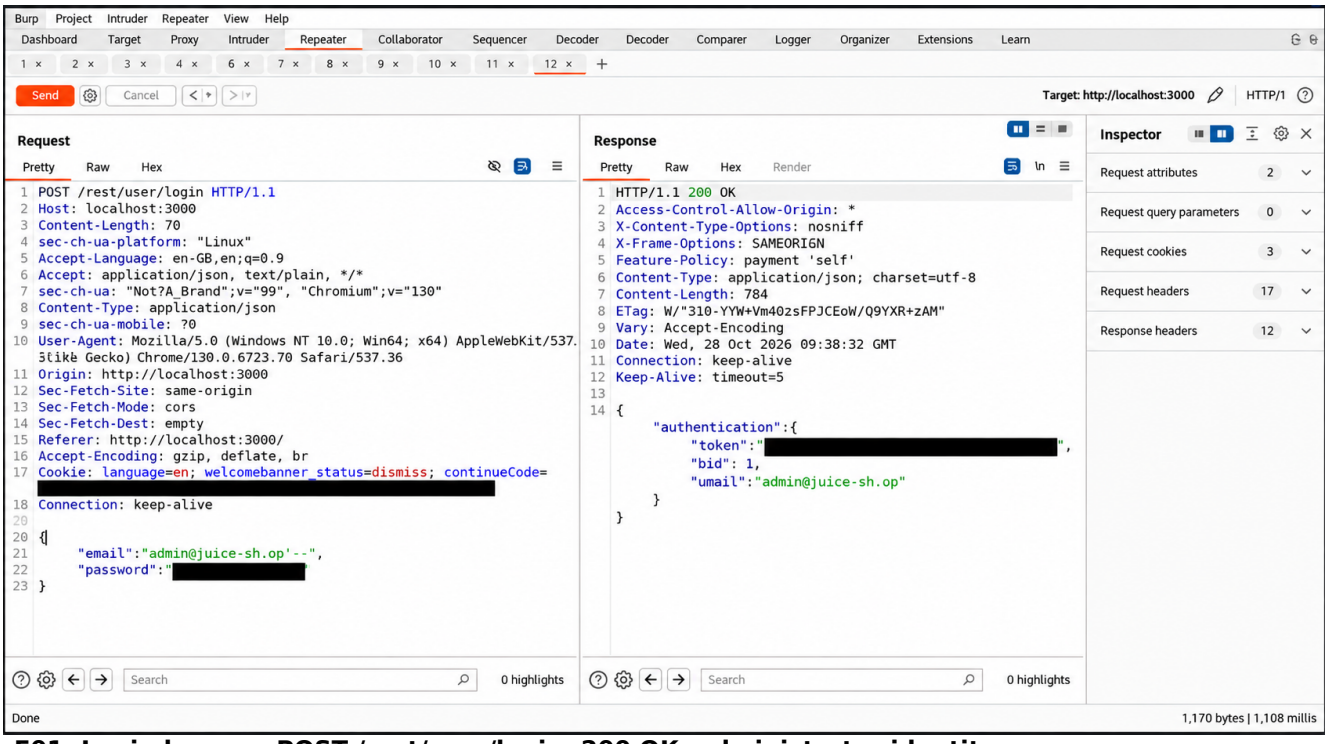
No application source code was reviewed. The precise SQL query and authorization middleware were not inspected. No sensitive product data was exposed in the basket response shown. No destructive operations or data exfiltration were performed.

Final conclusion

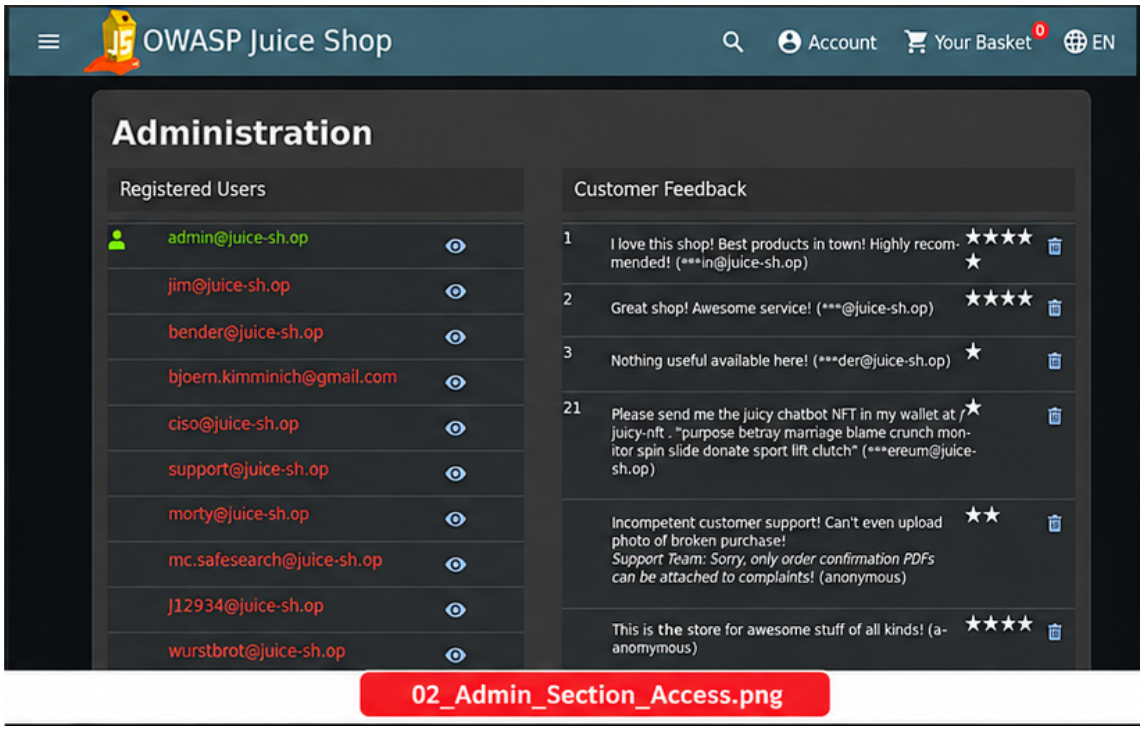
The assessment supports a confirmed SQL injection authentication bypass with subsequent administrator access. The cross-user basket read warrants verification of token continuity and ownership policy before being described as conclusively exploited.

10. Evidence Appendix — Authentication and Privileged Access

The following are original redacted screenshots supplied for the assignment. They are not AI-generated reconstructions.



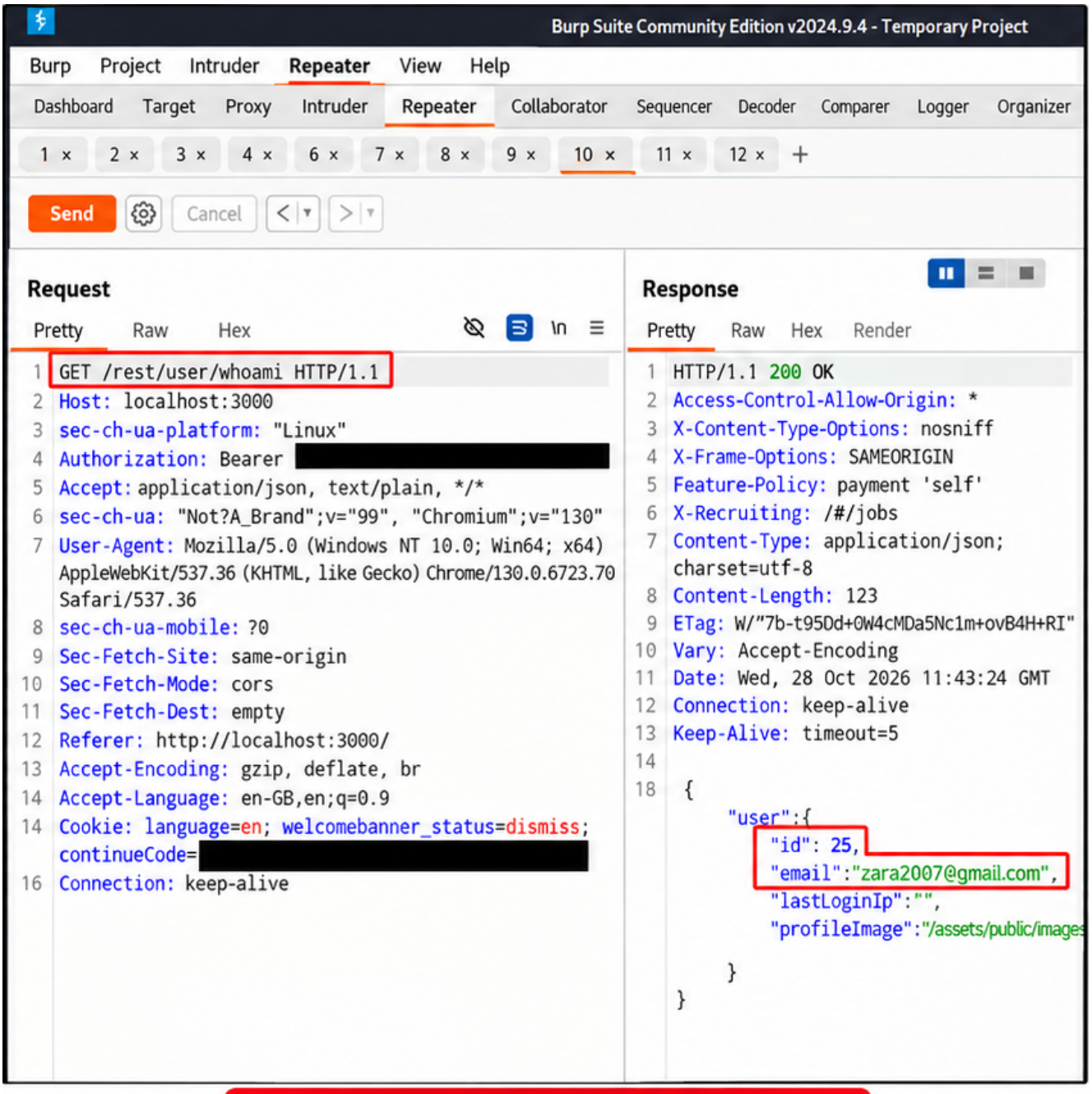
E01. Login bypass: POST /rest/user/login; 200 OK; administrator identity.



E02. Administration interface after administrator authentication.

11. Evidence Appendix — Regular-User Identity

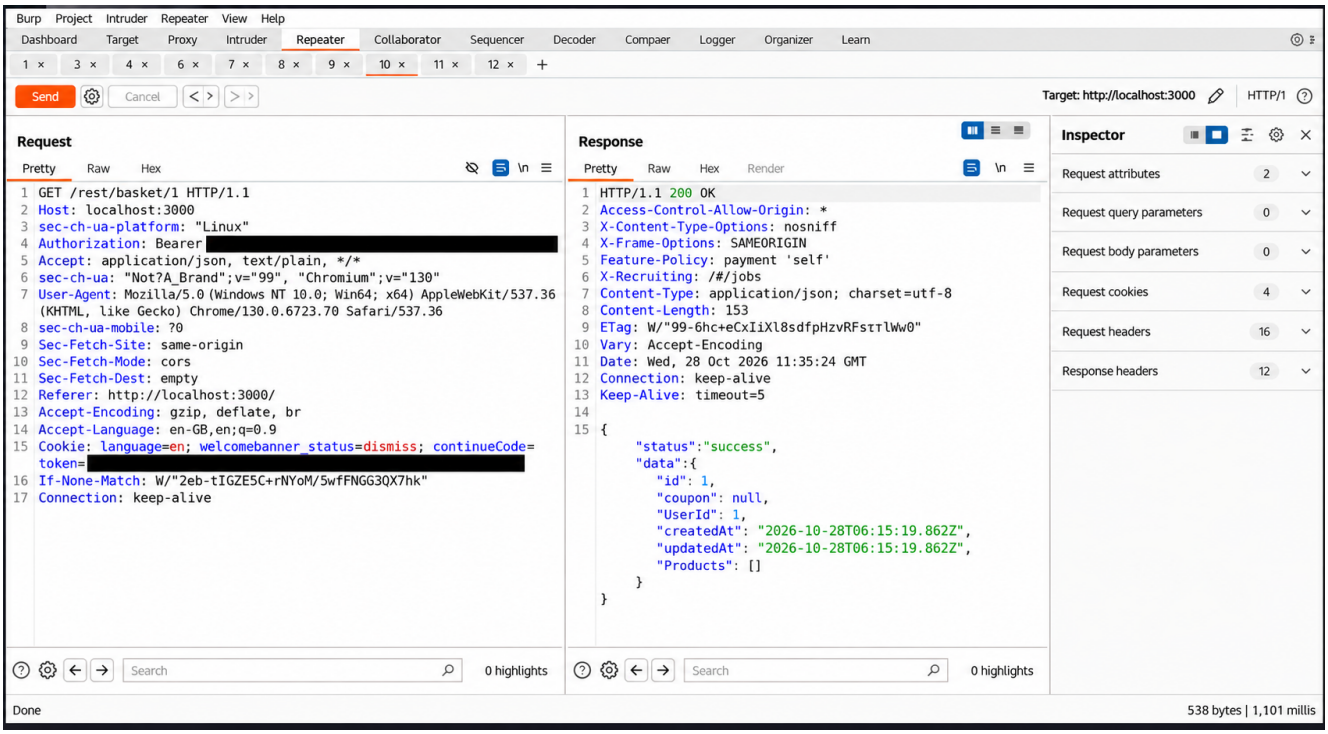
E03 documents the ordinary user identity. This image alone does not demonstrate a cross-user authorization defect.



E03. GET /rest/user/whoami returns user ID 25 (Zara).

12. Evidence Appendix — Basket Object Response

E04 documents the basket lookup. Compare this request against the original unredacted E03 request locally to verify identical authentication context.



E04. GET /rest/basket/1 returns basket ID 1, owner UserId 1, HTTP 200.